

EXAM PLATFORM

Production-Grade Online Examination System

JEE / NEET / CUET Scale — 1M+ Concurrent Users

Version	1.0.0
Date	June 2026
Stack	Spring Boot 3.2 · Kafka · Redis · PostgreSQL · MongoDB
Infra	Docker · Kubernetes (Azure AKS) · Cloudflare CDN

1. System Overview

The Exam Platform is a **production-grade, microservices-based online examination system** designed to handle **1 million+ concurrent users**. It supports real-time test taking, massive concurrency, high availability (99.99%), fault tolerance, and strict anti-cheating mechanisms. The architecture is inspired by systems like JEE Main, NEET, CUET, and TCS iON.

User Roles

Role	Description
Student	Test taker — registers, enrolls in exams, takes the test in real time
Admin	Exam creator — designs papers, manages questions and slots
Super Admin	Platform monitor — views real-time dashboards and analytics

2. High-Level Architecture

The system follows an **event-driven, microservices architecture**. All client traffic enters through a single **API Gateway** (Spring Cloud Gateway) which handles JWT validation, rate limiting, and routing. Services are stateless and communicate asynchronously via **Apache Kafka**. Answer state is maintained in **Redis** (hot path) and durably persisted to **MongoDB** (warm path) every 15 seconds.

Layer	Component	Technology
Entry	API Gateway	Spring Cloud Gateway + JWT Filter
Auth	User Service	Spring Boot · Argon2id · RS256 JWT
Exam Mgmt	Exam Service	Spring Boot · PostgreSQL · MongoDB
Real-time	Test Engine	Spring Boot · STOMP WebSocket · Redis
Evaluation	Result Service	Spring Boot · PostgreSQL window functions
Messaging	Event Bus	Apache Kafka (3 brokers, rf=3)
Cache	Session Store	Redis 7 (AOF persistence)
Relational DB	Structured Data	PostgreSQL 16 (1 primary + 3 replicas)
Document DB	Questions/Sessions	MongoDB 7 (3-shard Atlas cluster)
CDN / WAF	Static Assets	Cloudflare Enterprise
Infra	Orchestration	Azure AKS + Docker + Kubernetes

3. Microservices Breakdown

Service	Port	Responsibility
api-gateway	8080	JWT validation · Rate limiting · Circuit breaking · Routing
user-service	8081	Registration · Login · OTP · JWT issuance · Refresh tokens
exam-service	8082	Exam CRUD · Question bank · Slot management · Enrollment · Paper generation
test-engine	8083	WebSocket sessions · Autosave · Anti-cheat · Timer enforcement
result-service	8084	Score evaluation · Rank computation · Percentile · Analytics export

Infrastructure Containers (Dev)

Container	Image	Port	Status
exam-postgres	postgres:16-alpine	5432	Healthy
exam-mongodb	mongo:7.0	27017	Healthy
exam-redis	redis:7.2-alpine	6379	Healthy
exam-kafka	confluentinc/cp-kafka:7.6.0	9092	Up
exam-zookeeper	confluentinc/cp-zookeeper:7.6.0	2181	Up
exam-kafka-ui	provectuslabs/kafka-ui:latest	9000	Up

4. API Design — Key Endpoints

Method	Endpoint	Service	Auth	Description
POST	/api/v1/auth/register	user-service	None	Register new student
POST	/api/v1/auth/login	user-service	None	Login, returns JWT
POST	/api/v1/auth/otp/send	user-service	None	Send OTP to phone
POST	/api/v1/auth/refresh	user-service	None	Refresh access token
POST	/api/v1/auth/logout	user-service	JWT	Blacklist token
POST	/api/v1/exams	exam-service	ADMIN	Create new exam
GET	/api/v1/exams/{id}	exam-service	JWT	Get exam details
POST	/api/v1/exams/{id}/publish	exam-service	ADMIN	Publish exam + trigger pre-scale
GET	/api/v1/exams/{id}/slots	exam-service	JWT	List available slots
POST	/api/v1/exams/{id}/enroll	exam-service	STUDENT	Enroll student in slot
GET	/api/v1/exams/{id}/paper/{sid}	exam-service	STUDENT	Get student's shuffled paper
POST	/api/v1/sessions/start	test-engine	STUDENT	Start or resume exam session
GET	/api/v1/sessions/{id}	test-engine	STUDENT	Get current session state
POST	/api/v1/sessions/{id}/answer	test-engine	STUDENT	Save single answer (HTTP fallback)
POST	/api/v1/sessions/{id}/submit	test-engine	STUDENT	Final exam submission
GET	/api/v1/results/{examId}/{stuid}	result-service	JWT	Get student result
GET	/api/v1/results/{examId}/leaderboard	result-service	JWT	Top-N leaderboard
POST	/api/v1/results/{examId}/compute	result-service	ADMIN	Trigger rank computation
GET	/api/v1/results/{examId}/export	result-service	ADMIN	Download results CSV

5. Database Schema

PostgreSQL Tables

Table	Key Columns	Notes
users	id (UUID PK), email, phone, password_hash, role, is_verified	Fixed 2id hashed passwords
exams	id, title, exam_type, status, start_time, end_time, duration	DRAFT → PUBLISHED → LIVE → COMPLETED
exam_sections	id, exam_id (FK), name, subject, max_questions, marks	Ordered sections per exam
exam_slots	id, exam_id (FK), slot_date, start_time, capacity, enrolled	Distributed lock on enrollment
enrollments	id, student_id, exam_id, slot_id (FK), roll_number, status	Unique constraint (student+exam)
results	id, enrollment_id, student_id, exam_id, total_score, rank	Valid for rank/percentile

MongoDB Collections

Collection	Key Fields	Purpose
questions	exam_type, subject, difficulty, type, options, correct_answer	Question bank — MCQ/Numerical/Subjective
answer_snapshots	session_id, student_id, exam_id, answers{}, time_remaining	Per-session save backup (flush every 15s)

6. Critical System Flows

6.1 Student Takes Exam

Step	Action	Component
1	Student logs in, receives RS256 JWT (15 min) + Refresh token (7 days)	user-service
2	Student fetches shuffled paper (deterministic seed = hash(studentId+examId))	exam-service
3	POST /sessions/start — session created in Redis + MongoDB baseline	test-engine
4	WebSocket (STOMP) connection established to /ws/exam	test-engine
5	Client sends heartbeat every 5s with answers + time remaining	test-engine
6	Server saves to Redis < 2ms, marks session dirty for MongoDB flush	test-engine
7	Scheduled job flushes dirty sessions to MongoDB every 15s	test-engine
8	Student submits — immediate MongoDB flush + Kafka event published	test-engine
9	Result service consumes EXAM_SUBMITTED event, scores answers	result-service
10	Admin triggers rank computation using PostgreSQL window functions	result-service

6.2 Session Resume After Disconnect

Step	Action
1	Client reconnects, calls GET /sessions/{sessionId}
2	test-engine checks Redis for session state
3	Redis miss? Load from MongoDB, recompute time remaining (now - startedAt)
4	Re-populate Redis with restored state + new TTL
5	Return full session state (answers, time remaining) to client
6	Client resumes from exact state — zero answer loss

7. Scaling Strategy

Scale Numbers

Metric	Target
Concurrent WebSocket sessions	1,000,000 (500 pods x 2,000 connections)
Answer save latency (p99)	< 500ms
Peak join rate	500,000 users/minute
Data loss guarantee	Zero (Redis AOF + Kafka rf=3)
Availability	99.99% (Azure Traffic Manager multi-region)
RTO (Recovery Time Objective)	< 2 minutes

Kubernetes HPA Configuration

Service	Min Pods	Max Pods	Scale Trigger
api-gateway	3	20	CPU > 70%
user-service	2	10	CPU > 70%
exam-service	2	10	CPU > 70%
test-engine	10	500	WebSocket connections > 1,800/pod
result-service	2	20	Kafka consumer lag > 1,000

8. Security Design

Area	Implementation
Authentication	RS256 JWT (15 min access + 7 day refresh) with Redis blacklist
Password Hashing	Argon2id (iterations=3, memory=64MB, parallelism=4)
OTP	6-digit TOTP, 5 min TTL, 3 attempts max, 30 min lockout in Redis
Rate Limiting	Spring Cloud Gateway RequestRateLimiter — per IP and per user
DDoS Protection	Cloudflare Enterprise WAF + Azure DDoS Standard
Transport	HTTPS everywhere — TLS 1.3 enforced at Cloudflare edge
Anti-Cheat	Tab switch (15pts), copy-paste (10pts), dev tools (25pts) risk scoring
Secrets	JWT keys injected via environment variables — never committed to git
Headers	X-Frame-Options: DENY, X-Content-Type-Options: nosniff, CSP

9. CI/CD Pipeline

The platform uses **GitHub Actions** for CI and **ArgoCD (GitOps)** for CD to Azure AKS. Every push triggers automated build, test, Docker image publish, and rolling deployment.

Stage	Tool	Action
Build	GitHub Actions	Maven build + unit tests for all 5 services
Containerise	Docker	Multi-stage build → distroless Java 21 image
Publish	Azure Container Registry	Push image with commit SHA tag
Deploy (Dev)	ArgoCD	Auto-sync on main branch push
Deploy (Prod)	ArgoCD	Manual approval gate → rolling update on AKS
Health Check	Kubernetes	Readiness + liveness probes on /actuator/health
Rollback	ArgoCD	One-click rollback to any previous image SHA

10. Local Quick Start

Prerequisites: Docker Desktop, OpenSSL, Java 21

Step	Command
1. Generate JWT keys	<pre>openssl genrsa -out private.pem 2048 openssl rsa -in private.pem -pubout -out public.pem</pre>
2. Create .env	<pre>JWT_PRIVATE_KEY_PEM=\$(base64 -w0 private.pem) JWT_PUBLIC_KEY_PEM=\$(base64 -w0 public.pem)</pre>
3. Start stack	<pre>cd infra/docker docker-compose --env-file ../../.env up -d</pre>
4. Verify health	<pre>curl http://localhost:8080/actuator/health</pre>
5. Register student	<pre>curl -X POST http://localhost:8080/api/v1/auth/register \ -H "Content-Type: application/json" \ -d '{"firstName":"Test","lastName":"User","email":"test@example.com","password":"Test@1234"}</pre>

Service URLs

Service	URL
API Gateway	http://localhost:8080
Kafka UI	http://localhost:9000
PostgreSQL	localhost:5432 (user: examuser / pass: exampass)
MongoDB	localhost:27017
Redis	localhost:6379